

SALESTORM — Security & Observability Design

1. Purpose

SALESTORM handles high-value transactions during a flash sale where thousands of customers compete for limited inventory.

The security and observability design ensures:

- * Only authorized users can access protected operations.
- * Customer and payment data are protected.
- * Malicious or excessive traffic is controlled.
- * Duplicate and fraudulent requests are detected.
- * Every critical transaction can be traced.
- * System failures can be detected quickly.
- * Inventory and payment inconsistencies generate immediate alerts.

2. Security Architecture

Users

|



CDN / WAF

|

├── DDoS Protection

├── Bot Protection

├── IP Filtering

└── Attack Detection

|



Load Balancer

|



API Gateway

|

|— Authentication

|— Authorization

|— Rate Limiting

|— Input Validation

|— Request ID

|



Application Services

|

|— Inventory

|— Payment

|— Order

|— User

|



Secure Database

3. Authentication

SALESTORM uses authentication to verify the identity of customers and administrators.

Example:

Customer



Login



Authentication Service



Access Token



API Gateway



Protected API

A token-based mechanism such as JWT/OAuth 2.0 can be used.

Protected APIs require a valid authentication token.

Example:

Authorization: Bearer <access-token>

4. Authorization

Authentication answers:

Who are you?

What are you allowed to do?

SALESTORM uses role-based access control.

Role	Example Permissions
-----	-----
Customer	Browse products, reserve, pay, view orders
Admin	Manage products and inventory
Operations	Monitor system and failures

| Support | View customer/order information |

A customer should never be able to directly modify inventory or payment status.

5. HTTPS & Secure Communication

All external communication uses HTTPS/TLS.

Client

|

| HTTPS



API Gateway

|

| Secure internal communication



Services

This protects:

- * Authentication tokens
- * Customer information
- * Order information
- * Payment-related communication

6. Payment Security

SALESTORM does not directly store sensitive card information.

Instead:

Customer



Payment Service



External Payment Gateway



Payment Token / Reference



SALESTORM

SALESTORM stores only the information required for transaction tracking, such as:

- * Payment ID
- * Order ID
- * Transaction reference
- * Payment status
- * Timestamp

Sensitive payment credentials remain with the payment provider.

7. API Security

Every API request passes through security checks.

Request



Authentication



Authorization



Input Validation



Rate Limiting



Business Logic

Invalid requests are rejected before reaching critical services.

Example:

Invalid quantity = -10



Validation



HTTP 400 Bad Request

8. Rate Limiting

Flash sales are vulnerable to:

- * Request flooding
- * Bots
- * Brute-force requests
- * API abuse
- * Automated Buy-Now attacks

SALESTORM uses rate limiting at the API Gateway.

Client



1000 requests/sec



API Gateway



Rate Limit

|— Allowed requests

└─ Excess requests → 429 Too Many Requests

This protects downstream services.

9. Idempotency Security

Duplicate requests can occur because of:

- * Double-clicking
- * Network retries
- * Client retries
- * Malicious repeated requests

Critical requests use an idempotency key.

Example:

POST /payments

Idempotency-Key: PAY-12345

If the same key is received again:

First request

↓

Payment created

Duplicate request

↓

Existing transaction returned

Result:

One business request → one payment transaction.

10. Input Validation

All incoming data is validated before processing.

Examples:

quantity > 0

product_id exists

valid customer ID

valid payment reference

valid coupon

valid request format

This protects against:

- * Invalid data
- * Injection attacks
- * Malformed requests
- * Unexpected application behaviour

11. SQL Injection Protection

Database queries should use parameterized queries or ORM mechanisms.

Instead of directly constructing SQL from user input:

User Input → Raw SQL use:

User Input



Validation



Parameterized Query



Database

This prevents malicious input from being interpreted as executable SQL.

12. Secrets Management

Sensitive configuration should never be hardcoded.

Examples of secrets:

- * Database passwords
- * API keys
- * Payment gateway credentials
- * JWT secrets
- * Encryption keys

Instead:

Application



Secrets Manager / Environment Configuration



Secret

Secrets must not appear in:

- * Source code
- * GitHub repositories
- * Logs
- * Frontend JavaScript
- * Error responses

13. Audit Logging

Critical business actions are recorded in audit logs.

Examples:

USER_LOGIN

RESERVATION_CREATED

RESERVATION_RELEASED

PAYMENT_INITIATED

PAYMENT_SUCCESS

PAYMENT_FAILED

ORDER_CREATED

ORDER_CANCELLED

ADMIN_INVENTORY_UPDATE

Example:

json

```
{  
  "event": "PAYMENT_SUCCESS",  
  "order_id": "ORD-1024",  
  "payment_id": "PAY-7821",  
  "timestamp": "2026-10-05T10:30:21Z"  
}
```

Audit logs help investigate security incidents and business disputes.

14. Security Headers

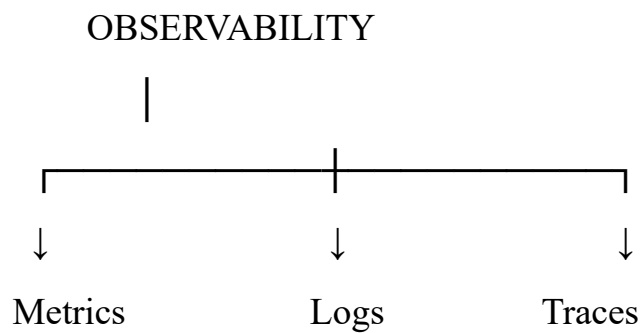
The API Gateway can enforce security headers such as:

- * HSTS
- * Content-Security-Policy
- * X-Content-Type-Options
- * Secure cookies
- * SameSite cookies

These reduce common web attack risks.

15. Observability Architecture

SALESTORM uses three major observability pillars:



Together they answer:

Metrics: What is happening?

Logs: What happened?

Traces: Where did it happen?

16. Metrics

The system continuously monitors important metrics.

Traffic Metrics:

- * Requests/sec
- * Active users
- * Requests accepted
- * Requests rejected
- * Rate-limit events

Performance Metrics:

- * P50 latency
- * P95 latency
- * P99 latency
- * Database latency
- * API response time

Business Metrics:

- * Reservation success rate
- * Reservation failure rate
- * Payment success rate
- * Payment failure rate
- * Order conversion rate
- * Inventory remaining

Reliability Metrics:

- * Service error rate
- * Retry count
- * Circuit-breaker state
- * Queue depth
- * Consumer lag
- * Dead-letter messages

17. Critical Inventory Metrics

Inventory receives special monitoring because it is the most important business invariant.

Original Stock = 100

Available = 30

Reserved = 20

Sold = 50

The system continuously verifies:

Available + Reserved + Sold = Original Stock

And critically:

Oversold Count = 0

If: Oversold Count > 0 a critical alert is generated immediately.

18. Structured Loggin

Logs use structured formats such as JSON rather than plain text.

Example:

```
{  
  "timestamp": "2026-10-05T10:30:20Z",  
  "service": "inventory-service",  
  "event": "RESERVATION_CREATED",  
  "product_id": "PROD-X",  
  "reservation_id": "RES-1001",  
  "quantity": 1,  
  "request_id": "REQ-5001"  
}
```

Structured logs make searching and analyzing production incidents easier.

19. Correlation ID

Every request receives a unique correlation/request ID.

Example:

REQ-5001

The same ID travels through services:

API Gateway

↓ REQ-5001

Checkout

↓ REQ-5001

Inventory

↓ REQ-5001

Payment

↓ REQ-5001

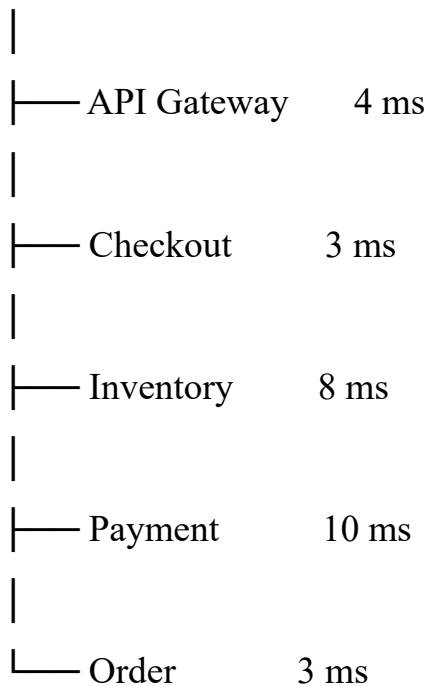
Order

Therefore, engineers can trace one customer's purchase across the entire system.

20. Distributed Tracing

A distributed tracing system records the complete journey of a request.

Request



Total: 28 ms

If latency suddenly becomes 200 ms, tracing can identify which service caused the slowdown.

21. P95 & P99 Latency

SALESTORM monitors tail latency rather than only average latency.

Example:

P50 = 12 ms

P95 = 28.7 ms

P99 = 45 ms

Why P95?

P95 means:> 95% of requests completed within this latency.

It helps identify whether a significant portion of customers are experiencing slow responses.

22. Alerting

The monitoring system generates alerts when predefined thresholds are crossed.

Alert	Severity
-----	-----
Oversold Count > 0	● Critical
Payment failure spike	● Critical
Database unavailable	● Critical
API error rate > threshold	● Critical
P95 latency high	● Warning
Queue backlog increasing	● Warning
Circuit breaker OPEN	● Warning
Reservation expiry backlog	● Warning

23. Security Monitoring

Security events are also monitored.

Examples:

Repeated failed login



Security Alert

Unusual request rate



Possible Bot / Abuse

Repeated Buy requests



Rate Limiting

Invalid authentication token



Request Rejected

Security logs can be correlated with application logs to investigate suspicious activity.

24. Monitoring Dashboard

The operations dashboard should display:

SALESTORM MONITOR		
Traffic	10,000 req/s	
P95 Latency	28.7 ms	
Error Rate	0.12%	
Inventory	100 → 0	
Reservations	100 successful	
Payments	95 successful	
Orders	95 confirmed	

Oversold	0	
Queue Lag	12 messages	
System	HEALTHY	

This gives the operations team a real-time view of system health.

25. Security & Observability During 10,000-User Attack

When 10,000 customers simultaneously click ****BUY NOW****:

10,000 Requests



WAF



Rate Limiting



API Gateway



Authentication



Inventory



Atomic Reservation

At the same time, monitoring records:

Requests = 10,000

Successful Reservations = 100

Rejected = 9,900

Oversold = 0

This provides both ****protection**** and ****evidence****.

26. Security Failure Scenarios

Scenario	Protection	
-----	-----	
DDoS/request flood	WAF + rate limiting	
Bot attack	WAF + rate limiting + behavioural controls	
Duplicate Buy request	Idempotency	
Duplicate payment	Idempotency + unique transaction reference	
Unauthorized inventory modification	RBAC	
SQL injection	Input validation + parameterized queries	
Credential leakage	Secrets management	
Data interception	HTTPS/TLS	
Suspicious activity	Security logging + alerts	
Service abuse	API Gateway throttling	

27. Observability Failure Scenarios

Problem	Detection	
-----	-----	
Inventory overselling	Inventory invariant alert	
Payment failures increase	Payment failure metric	
Order Service unavailable	Health check + error rate	
Queue backlog	Queue-depth alert	
Slow database	DB latency metric	

High API latency	P95/P99 alert	
Failed event processing	Retry/DLQ metrics	
Service crash	Health check	
Network failure	Timeout/error metrics	

28. Security Principles

SALESTORM follows:

1. Least privilege
2. Defense in depth
3. Zero trust between sensitive operations
4. Secure communication
5. Input validation
6. Strong authentication
7. Role-based authorization
8. Idempotency for critical operations
9. Secrets protection
10. Complete auditability

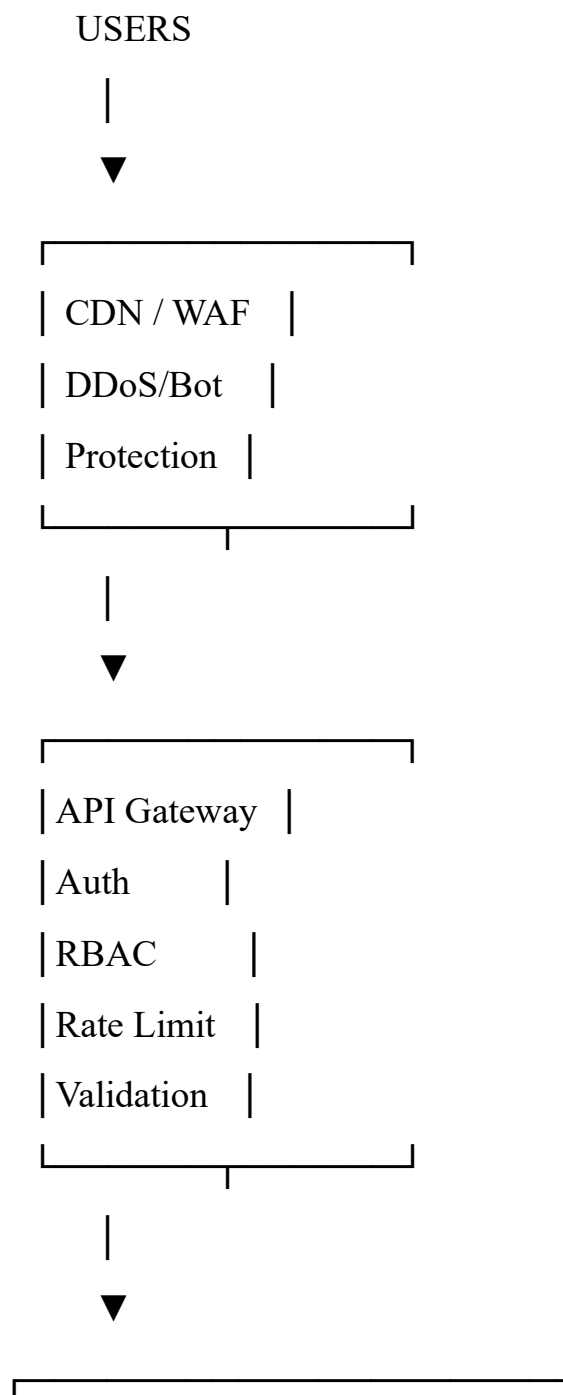
29. Observability Principles

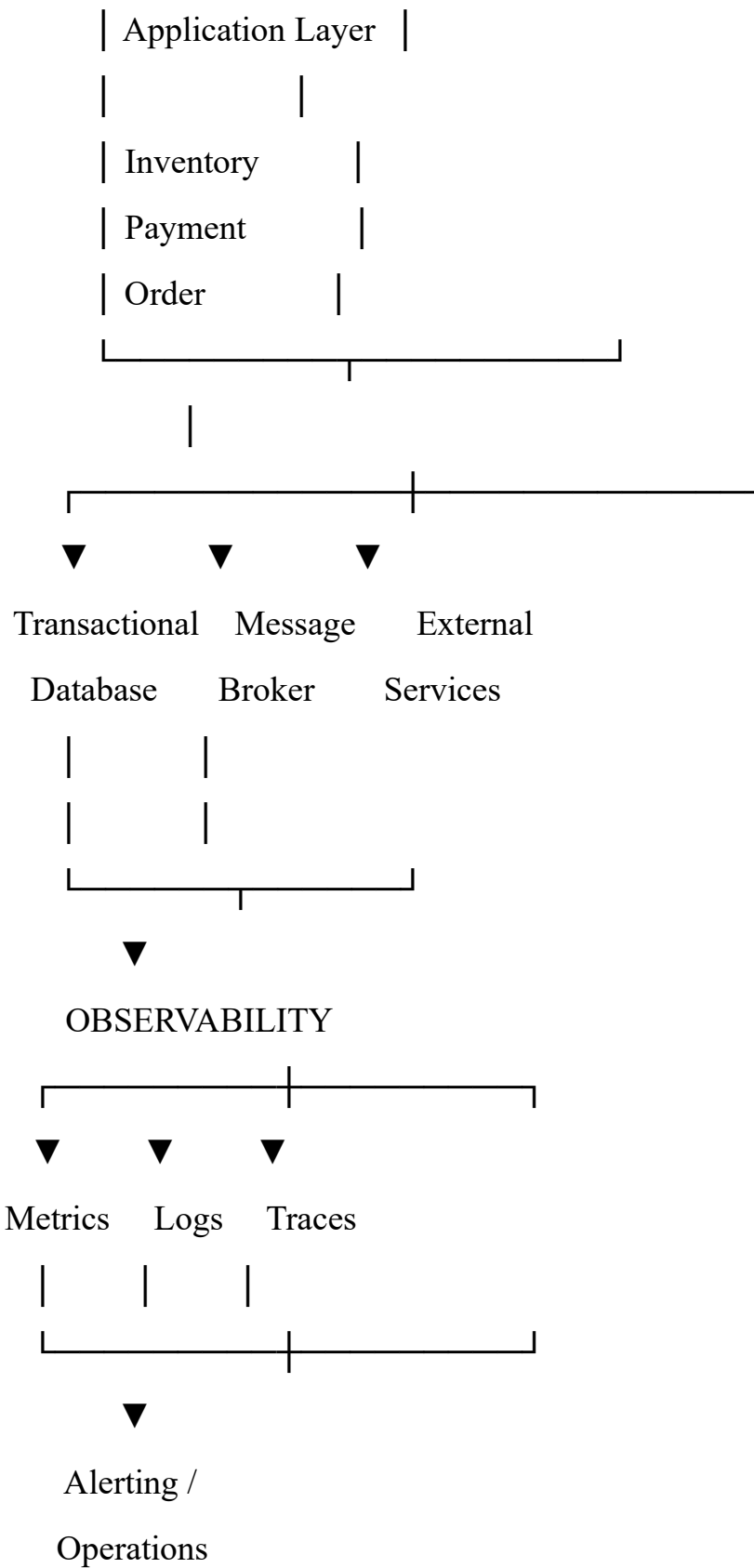
SALESTORM follows:

1. Measure every critical workflow
2. Use structured logs
3. Use correlation IDs
4. Trace distributed transactions
5. Monitor business invariants
6. Monitor infrastructure health

7. Alert on abnormal behaviour
8. Track queue and retry behaviour
9. Monitor payment and inventory separately
10. Make failures diagnosable

30. Final Security & Observability Design





Final Design Statement

SALESTORM protects the system using defense-in-depth security, authentication, authorization, WAF, rate limiting, input validation, idempotency and secure payment handling. Its observability layer combines metrics, structured logs and distributed tracing to provide complete visibility into traffic, inventory, payment and order workflows. Critical business invariants such as `OVERSOLD = 0`, payment failures and queue backlogs are continuously monitored and trigger alerts, allowing the engineering team to detect, diagnose and recover from failures quickly.